

Chap 10. Standard Input/output

Contents

01 Standard Input/output

02 Memory Address

03 Pointer

C User Input

C User Input

■ User Input

- You have already learned that `printf()` is used to output values in C.
- To get user input, you can use the `scanf()` function:

```
// Create an integer variable that will store
the number we get from the user
int myNum;

// Ask the user to type a number
printf("Type a number: \n");

// Get and save the number the user types
scanf("%d", &myNum);

// Output the number the user typed
printf("Your number is: %d", myNum);
```

C User Input

https://github.com/kkh3363/C_Lang

■ Multiple Inputs

- The scanf() function also allow multiple inputs (an integer and a character in the following example):

```
// Create an int and a char variable
int myNum;
char myChar;

// Ask the user to type a number AND a character
printf("Type a number AND a character and press enter:
\n");

// Get and save the number AND character the user
types
scanf("%d %c", &myNum, &myChar);

// Print the number
printf("Your number is: %d\n", myNum);

// Print the character
printf("Your character is: %c\n", myChar);
```

C User Input

■ Take String Input

- You can also get a string entered by the user:

```
// Create a string
char firstName[30];

// Ask the user to input some text
printf("Enter your first name: \n");

// Get and save the text
scanf("%s", firstName);

// Output the text
printf("Hello %s", firstName);
```

C User Input

https://github.com/kkh3363/C_Lang

■ Limitation

- the `scanf()` function has some limitations: it considers space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words). For example:

```
char fullName[30];

printf("Type your full name: \n");
scanf("%s", &fullName);

printf("Hello %s", fullName);
// Type your full name: John Doe
// Hello John
```

C User Input

https://github.com/kkh3363/C_Lang

■ String get

- when working with strings, we often use the `fgets()` function to read a line of text. Note that you must include the following arguments: the name of the string variable, `sizeof(string_name)`, and `stdin`:
- Syntax
 - `char *fgets(char *line, int maxline, FILE *fp)`

```
char fullName[30];

printf("Type your full name: \n");
fgets(fullName, sizeof(fullName), stdin);

printf("Hello %s", fullName);

// Type your full name: John Doe
// Hello John Doe
```


I/O function

https://github.com/kkh3363/C_Lang

- A console comprises the VDS and the keyboard. The Console Input and Output functions can be classified into two categories:
 - **Formatted console I/O functions:** These functions allow the user to format the input from the keyboard and the output displayed in the desired manner.
 - **Unformatted console I/O functions:** These functions do not allow the user this feature.

Formatted Functions		
Type	Input	Output
char	scanf()	printf()
int	scanf()	printf()
float	scanf()	printf()
string	scanf()	printf()

Unformatted Functions		
Type	Input	Output
char	getch() getche() getchar()	putch() putchar()
int	-	-
float	-	-
string	gets()	puts()

Codinggeek.com

I/O function

https://github.com/kkh3363/C_Lang

■ Unformatted Console I/O functions

Functions	Description
<code>getch()</code>	This function returns the character that was typed last or was typed most recently. It does not display(echo) the character on the screen. It is present in the header file <code><conio.h></code> .
<code>getche()</code>	This function is the same as <code>getch()</code> . The only difference is that it echoes(displays) the most recently used character. It is also present in <code><conio.h></code> .
<code>getchar()</code>	<code>getchar()</code> is a macro that works in a similar manner as <code>getch()</code> and also displays(echoes) the character but it needs the user to press the Enter key after the character. It is present in the header file <code><stdio.h></code> .
<code>fgetchar()</code>	<code>fgetchar()</code> works in the same manner as <code>getchar()</code> . But <code>fgetchar()</code> is a function. It is also present in <code><stdio.h></code> .
<code>putch()</code>	This function prints a single character on the console.
<code>putchar()</code>	This function works in the same way as <code>putch()</code> .
<code>fputchar()</code>	This function works in the same way as <code>putch()</code> .
<code>gets()</code>	It takes a string input(single-word or multi-word) from the user. It terminates when the Enter key is pressed.
<code>puts()</code>	It is used to print the string to the console.

C Memory

Memory Address

■ Memory Address

- When a variable is created in C, a memory address is assigned to the variable.
- The memory address is the location of where the variable is stored on the computer.
- When we assign a value to the variable, it is stored in this memory address.
- To access it, use the reference operator (&), and the result represents where the variable is stored:

```
int myAge = 43;  
printf("%p", &myAge); // Outputs 0x7ffe5367e044
```

Memory Address

■ Memory Address

- The memory address is in hexadecimal form (0x..). You will probably not get the same result in your program, as this depends on where the variable is stored on your computer.
- You should also note that ***&myAge*** is often called a "pointer". A pointer basically stores the memory address of a variable as its value. To print pointer values, we use the **%p** format specifier.

Pointers

■ Pointers

- we can get the memory address of a variable with the reference operator **&**.

```
int myAge = 43; // an int variable

printf("%d", myAge); // Outputs the value of myAge (43)
printf("%p", &myAge); // Outputs the memory address of myAge (0x7ffe5367e044)
```

- A pointer is a variable that stores the memory address of another variable as its value.
- A pointer variable points to a data type (like int) of the same type, and is created with the ***** operator.
- The address of the variable you are working with is assigned to the pointer:

Pointers

■ Example

- Create a pointer variable with the name ptr, that points to an int variable (myAge). Note that the type of the pointer has to match the type of the variable you're working with (int in our example).
- Use the & operator to store the memory address of the myAge variable, and assign it to the pointer.
- Now, ptr holds the value of myAge's memory address.

```
int myAge = 43;           // An int variable
int* ptr = &myAge;        // A pointer variable,
                           // with the name ptr, that stores the address of
                           // myAge

// Output the value of myAge (43)
printf("%d\n", myAge);

// Output the memory address of myAge
// (0x7ffe5367e044)
printf("%p\n", &myAge);

// Output the memory address of myAge with the
// pointer (0x7ffe5367e044)
printf("%p\n", ptr);
```

Pointers

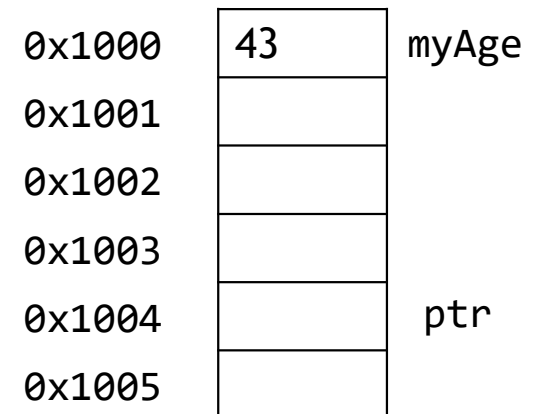
■ Dereference

- we used the pointer variable to get the memory address of a variable (used together with the & reference operator).
- You can also get the value of the variable the pointer points to, by using the * operator (the dereference operator):

```
int myAge = 43;      // Variable declaration
int* ptr = &myAge;   // Pointer declaration

// Reference: Output the memory address of myAge
// with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);

// Dereference: Output the value of myAge with the pointer (43)
printf("%d\n", *ptr);
```



0x1000

Pointers

■ Pointers & Arrays

- You can also use pointers to access arrays. Consider the following array of integers:

```
int myNumbers[4] = {25, 50, 75, 100};  
int i;  
  
for (i = 0; i < 4; i++) {  
    printf("%d\n", myNumbers[i]);  
    printf("%p\n", &myNumbers[i]);  
}
```

Pointers

■ Pointers & Arrays

- This basically means that we can work with arrays through pointers!
- How? Since myNumbers is a pointer to the first element in myNumbers, you can use the * operator to access it:

```
int myNumbers[4] = {25, 50, 75, 100};  
  
// Get the value of the first element in myNumbers  
printf("%d", *myNumbers);
```

Pointers

- To access the rest of the elements in myNumbers, you can increment the pointer/array (+1, +2, etc):

```
int myNumbers[4] = {25, 50, 75, 100};  
  
// Get the value of the second element in myNumbers  
printf("%d\n", *(myNumbers + 1));  
  
// Get the value of the third element in myNumbers  
printf("%d", *(myNumbers + 2));
```

```
int myNumbers[4] = {25, 50, 75, 100};  
int *ptr = myNumbers;  
int i;  
  
for (i = 0; i < 4; i++) {  
    printf("%d\n", *(ptr + i));  
}
```

Pointers

■ Pointer Arithmetic

- Pointer arithmetic means changing the value of a pointer to make it point to a different element in memory.
- Like we saw on the previous page, array elements are stored next to each other in memory. So if a pointer points to one element, adding 1 moves it to the next one:

```
int myNumbers[4] = {25, 50, 75, 100};
int *p = myNumbers; // points to myNumbers[0]

printf("%d\n", *p);           // 25
printf("%d\n", *(p + 1));    // 50
printf("%d\n", *(p + 2));    // 75
printf("%d\n", *(p + 3));    // 100
```

```
int myNumbers[3] = {10, 20, 30};
int *p = myNumbers; // myNumbers[0]

printf("%d\n", *p); // 10
p++;                // move to myNumbers[1]
printf("%d\n", *p); // 20
p--;                // back to myNumbers[0]
printf("%d\n", *p); // 10

p += 2;             // jump to myNumbers[2]
printf("%d\n", *p); // 30
```

Pointers

■ Pointer Subtraction (Distance)

```
int myNumbers[5] = {10, 20, 30, 40, 50};  
int *start = &myNumbers[1]; // points to 20  
int *end = &myNumbers[4];    // points to 50  
  
printf("%ld\n", end - start); // 3 elements apart
```

- start points to the second element (20).
- end points to the fifth element (50).
- end - start gives the number of elements between them: 3.

Pointers

■ Pointer Arithmetic Depends on Type

- When you add 1 to a pointer, it moves forward by the size of the thing it points to - not just by 1 byte.
- For example:
 - An `int*` pointer moves by the size of an integer (usually 4 bytes).
 - A `char*` pointer moves by the size of a character (1 byte).
- So if both pointers start at memory address **1000**:
 - `int*` → `p + 1` would move to address **1004**
 - `char*` → `p + 1` would move to address **1001**

```
int myNumbers[2] = {1, 2};
char letters[] = "Hi"; // 'H', 'i', '\0'

int *pi = myNumbers; // int pointer
char *pc = letters; // char pointer

printf("%p\n", (void*)pi);
printf("%p\n", (void*)(pi + 1)); // moves by
sizeof(int) (4 bytes)
printf("%p\n", (void*)(pi + 2)); // moves by
sizeof(int) (4 bytes)

printf("%p\n", (void*)pc);
printf("%p\n", (void*)(pc + 1)); // moves by 1
byte
```

Pointers

■ Looping with Pointers

- In the previous chapter, you learned how to loop through an array using `*(ptr + i)`.
- Now let's look at another way - by moving the pointer itself inside the loop. Each time the pointer is increased (`p++`), it moves to the next element in memory:

```
int myNumbers[4] = {25, 50, 75, 100};
int *p = myNumbers;    // start of array

for (int i = 0; i < 4; i++) {
    printf("%d\n", *p);
    p++; // move to next element
}
```

- `*p` gives the current element value.
- `p++` moves the pointer to the next element in the array.
- No array index (`i`) is needed - the pointer keeps track of the position.

Pointers

■ Pointer to Pointer

- You can also have a pointer that points to another pointer. This is called a pointer to pointer (or "double pointer").
- It might sound confusing at first, but it's just one more level of indirection: a pointer that stores the *address of another pointer*.

```
int myNum = 10;           // normal variable
int *ptr = &myNum;        // pointer to int
int **pptr = &ptr;        // pointer to pointer

printf("myNum = %d\n", myNum);
printf("*ptr = %d\n", *ptr);
printf("**pptr = %d\n", **pptr);
```


Pointers

- Changing Values Through a Pointer to Pointer
 - Since `**pptr` accesses the original variable, you can use it to change the value of the variable too:

```
int myNum = 5;
int *ptr = &myNum;
int **pptr = &ptr;

**pptr = 20; // changes myNum

printf("myNum = %d\n", myNum); // prints 20
```

Pointers

■ Summary

- A pointer to pointer stores the address of another pointer.
- *ptr gives the value of a variable.
- **pptr(double pointer) gives the same value by following two levels of indirection.
- They can be useful when passing pointers to functions or working with complex data structures.

Pointers

https://github.com/kkh3363/C_Lang

■ Example

```
#include <stdio.h>
int main()
{
    int* pc, c;

    c = 22;
    printf("Address of c: %p\n", &c);
    printf("Value of c: %d\n\n", c); // 22

    pc = &c;
    printf("Address of pointer pc: %p\n", pc);
    printf("Content of pointer pc: %d\n\n", *pc); // 22

    c = 11;
    printf("Address of pointer pc: %p\n", pc);
    printf("Content of pointer pc: %d\n\n", *pc); // 11

    *pc = 2;
    printf("Address of c: %p\n", &c);
    printf("Value of c: %d\n\n", c); // 2
    return 0;
}
```

Thank You !